

Computation with occasionally binding constraints

In these notes, I show how we can solve for the general equilibrium using the **Projection Method**.

Any dynamic model with a recursive representation is characterized by a system of difference equations (of first or higher order). This means that the equations describing the equilibrium contain variables at different times t . If the system contains variables at t and $t + 1$, it is a first-order system. If it contains variables at t , $t + 1$, and $t + 2$, it is a second-order system, and so on. For the specific model we are trying to solve, the dynamic system is given by the following equations:

$$w_t L_t + B_t + D_t = C_t + \frac{B_{t+1}}{R_t} + T_t \quad (1)$$

$$U_c(C_t, L_t) w_t = U_\ell(C_t, L_t) \quad (2)$$

$$U_c(C_t, L_t) = \beta R_t E_t U_c(C_{t+1}, L_{t+1}) \quad (3)$$

$$\varphi(D_t) + B_t + K_{t+1} = F(z_t, K_t, L_t) - w_t L_t + \frac{(1 + \tau) B_{t+1}}{R_t} \quad (4)$$

$$\xi_t K_{t+1} \geq \frac{(1 + \tau) B_{t+1}}{R_t} \quad (5)$$

$$F_\ell(z_t, K_t, L_t) = (1 + \mu_t) w_t \quad (6)$$

$$\frac{(1 + \tau)(1 - \mu_t) U_c(C_t, L_t)}{\varphi(D_t) R_t} = \beta E_t \frac{U_c(C_{t+1}, L_{t+1})}{\varphi(D_{t+1})} \quad (7)$$

$$\frac{(1 - \mu_t \xi_t) U_c(C_t, L_t)}{\varphi(D_t)} = \beta E_t \frac{U_c(C_{t+1}, L_{t+1}) F_k(z_{t+1}, K_{t+1}, L_{t+1})}{\varphi(D_{t+1})} \quad (8)$$

$$T_t = \left(\frac{1}{R_t} - \frac{1 + \tau}{R_t} \right) B_{t+1} \quad (9)$$

Equations (1)–(9) are obtained after imposing aggregate consistency conditions (general equilibrium). The system contains variables at times t and $t + 1$ (it is a first-order system) and has the following properties:

- State variables: $\mathbf{s}_t = (z_t, \xi_t, K_t, B_t)$.
- Control variables: $\mathbf{x}_t = (C_t, L_t, w_t, R_t, D_t, T_t, \mu_t, B_{t+1}, K_{t+1})$.
- Number of endogenous variables and equations: 9.
- Solution: $\mathbf{x}_t = g(\mathbf{s}_t)$.

The objective is to find the solution $\mathbf{x}_t = g(\mathbf{s}_t)$. Unfortunately, the function $g(\mathbf{s}_t)$ is highly nonlinear and cannot be derived analytically. We there-

fore seek an approximate solution $\hat{g}(\mathbf{s}_t)$ that is as close as possible to the true function $g(\mathbf{s}_t)$.

Before proceeding, it is useful to highlight in the above system the terms that contain control variables at $t + 1$:

$$w_t L_t + B_t + D_t = C_t + \frac{B_{t+1}}{R_t} + T_t \quad (10)$$

$$U_c(C_t, L_t) w_t = U_\ell(C_t, L_t) \quad (11)$$

$$U_c(C_t, L_t) = \beta R_t E_t \underbrace{U_c(C_{t+1}, L_{t+1})}_{\Phi(\mathbf{s}_{t+1})} \quad (12)$$

$$\varphi(D_t) + B_t + K_{t+1} = F(z_t, K_t, L_t) - w_t L_t + \frac{(1 + \tau) B_{t+1}}{R_t} \quad (13)$$

$$\xi_t K_{t+1} \geq \frac{(1 + \tau) B_{t+1}}{R_t} \quad (14)$$

$$F_\ell(z_t, K_t, L_t) = (1 + \mu_t) w_t \quad (15)$$

$$\frac{(1 + \tau)(1 - \mu_t) U_c(C_t, L_t)}{\varphi(D_t) R_t} = \beta E_t \underbrace{\frac{U_c(C_{t+1}, L_{t+1})}{\varphi(D_{t+1})}}_{\Psi(\mathbf{s}_{t+1})} \quad (16)$$

$$\frac{(1 - \mu_t \xi_t) U_c(C_t, L_t)}{\varphi(D_t)} = \beta E_t \underbrace{\frac{U_c(C_{t+1}, L_{t+1}) F_k(z_{t+1}, K_{t+1}, L_{t+1})}{\varphi(D_{t+1})}}_{\Gamma(\mathbf{s}_{t+1})} \quad (17)$$

$$T_t = \left(\frac{1}{R_t} - \frac{1 + \tau}{R_t} \right) B_{t+1} \quad (18)$$

In a recursive system, the control variables at $t + 1$ are determined by the states at $t + 1$. In particular, D_{t+1} , C_{t+1} , and L_{t+1} are determined by the states at $t + 1$. Thus, the functions $\Phi(\mathbf{s}_{t+1})$, $\Psi(\mathbf{s}_{t+1})$, and $\Gamma(\mathbf{s}_{t+1})$ depend only on the states at $t + 1$, as indicated above.

The states at $t + 1$ are either exogenous— z_{t+1} and ξ_{t+1} —or determined by the control variables at time t — K_{t+1} and B_{t+1} . This implies that, if we knew the functions $\Phi(\mathbf{s}_{t+1})$, $\Psi(\mathbf{s}_{t+1})$, and $\Gamma(\mathbf{s}_{t+1})$, we could solve the above system exactly for all control variables $\mathbf{x}_t$ given the current states $\mathbf{s}_t$. The problem is that we do not know these functions, and the numerical procedure must also solve for them. Here is how we proceed:

1. Assume that $\Phi(\mathbf{s}_{t+1})$, $\Psi(\mathbf{s}_{t+1})$, and $\Gamma(\mathbf{s}_{t+1})$ take specific functional forms characterized by a few parameters.
2. After guessing the parameter values of these functions, solve the dynamic system for a given set of states $\mathbf{s}_t$.
3. After finding the solution (which is conditional on the guessed functions $\Phi(\mathbf{s}_{t+1})$, $\Psi(\mathbf{s}_{t+1})$, and $\Gamma(\mathbf{s}_{t+1})$), update the guessed parameters of these functions by checking conditions that inform us about the accuracy of our guesses. If the functions are characterized by N parameters, we need to check N conditions.
4. Iterate until the accuracy conditions are sufficiently small (convergence).

Here is a more detailed description of the steps:

STEP 1: Choose a grid for the states $\mathbf{s} = (z, \xi, K, B)$.

- Use subscripts i, j, k , and b for the grid points of the four states: z_i, ξ_j, K_k, B_b .

STEP 2: Parameterize the functions $\Phi(\mathbf{s})$, $\Psi(\mathbf{s})$, and $\Gamma(\mathbf{s})$.

- Assign a value for each combination of grid points for the states. In the computer, the parameterized functions are four-dimensional arrays: $\Phi_{i,j,k,b}$, $\Psi_{i,j,k,b}$, and $\Gamma_{i,j,k,b}$. Values outside the grid are obtained by linear interpolation.

STEP 3: Solve the dynamic system to find $\mathbf{x}$ at each grid point.

- When solving the system, we must check the **Kuhn–Tucker** conditions. This is done by first solving the system under the assumption that the borrowing constraint is binding. As part of

the solution, we also obtain the multiplier μ_t . If this is positive, then the borrowing constraint is indeed binding. If μ_t is negative, then the constraint is not binding; in this case, we set $\mu_t = 0$ and resolve the system without the borrowing constraint.

- The solution is obtained only at the grid points: $\mathbf{x}_{i,j,k,b}$.

STEP 4: Use the solution to update the guesses for $\Phi(\mathbf{s})$, $\Psi(\mathbf{s})$, and $\Gamma(\mathbf{s})$.

- We can use the values of $\Phi_{i,j,k,b}$, $\Psi_{i,j,k,b}$, and $\Gamma_{i,j,k,b}$ at t to generate guesses for their values at $t + 1$.

Running the codes

The codes used to solve the model are written in Fortran, while the code used to construct the graphs is written in GAUSS. These are the code files:

- `MainProg.f90` – File containing the Fortran code for the main program.
- `Param.f90` – File containing the predefined parameters written in Fortran.
- `Routines.f90` – File containing various functions written in Fortran.
- `Simulation.Procedures.f90` – File containing the Fortran subroutines that simulate the model after a solution has been found (the function $g(\mathbf{s})$).
- `MakePlot.gau` – GAUSS code that uses the data generated by the Fortran code to construct the simulation graphs.

These are the instructions to run the above codes:

1. Save the first four files (`MainProg.f90`, `Param.f90`, `Routines.f90`, `Simulation.Procedures.f90`) in a directory.
2. In the same directory where you saved the above files, create a sub-directory named `Simulation`. In this directory, the Fortran code will save various files containing the data generated through the simulation of the model. These data files will be used to construct the graphs.

3. In the subdirectory **Simulation**, save the GAUSS code **MakePlot.gau**.
4. Compile the Fortran file **MainProg.f90**.
5. Run (execute) the executable file created by compiling **MainProg.f90**.
When the execution finishes, the model will have been run and simulated. At this point, you will see new files stored in the subdirectory **Simulation**.
6. To construct the graphs, run the GAUSS program **MakePlot.gau** that you previously saved in subdirectory **Simulation**.